RAPPORT DE REVUE TECHNIQUE

PROJECT RESILIENCE-ALPHA

Dépôt : pierre-mcd38/RESILIENCE-ALPHA | Date : 21 octobre 2026

Destinataire : responsable hiérarchique / responsable technique

Diffusion recommandée : interne - informations relatives à l'architecture et à ses faiblesses.

1. Synthèse exécutive

Le dépôt présente un démonstrateur cohérent dans son intention : paiement fictif, détection d'anomalies, remédiation limitée et preuve transactionnelle de reprise. Plusieurs mécanismes existent effectivement dans le code : montants en centimes, écritures SQLite atomiques, idempotence, contrôle de permissions, signatures HMAC et actions d'exécution limitées.

La principale réserve concerne la correspondance entre la documentation, les essais locaux et le déploiement Docker annoncé. La revue identifie des écarts susceptibles d'empêcher un démarrage fonctionnel, de perdre les transactions au remplacement du conteneur API ou de laisser les secrets par défaut actifs. Le fonctionnement local ne suffit pas à démontrer que la même chaîne fonctionne en conteneurs.

Avis : conserver le statut de démonstrateur de laboratoire et ne pas déclarer la pile Docker validée avant une recette dédiée. Cette revue ne conclut ni à une compromission, ni à une perte de données effectivement survenue.

2. Méthode et limites

Revue statique en lecture seule des principaux services, de Docker Compose, des Dockerfiles ciblés, des quatre fichiers de tests, des scripts de secrets et de lancement, des configurations d'observabilité, d'un playbook de déploiement et des documents de gouvernance et de recette. Aucune écriture, aucun commit, aucune modification du dépôt, aucun lancement de scénario offensif et aucune exécution des tests n'ont été réalisés par l'auteur de la revue.

Les résultats PASS cités sont des éléments versionnés produits antérieurement, non des résultats reproduits pendant cette revue. Certains fichiers volumineux ont été retournés partiellement, notamment run_live_platform.py, remediator/app.py et build_deliverables.py. Le sujet PDF, les binaires PDF/PPTX/ZIP et le modèle sérialisé n'ont pas fait l'objet d'une inspection exhaustive. La revue ne constitue donc ni un audit ligne à ligne exhaustif, ni une certification de sécurité ou de conformité.

Les chemins et empreintes SHA de fichiers en annexe permettent de retrouver les preuves consultées. Aucun identifiant global de commit n'a été figé ; les conclusions décrivent les versions de fichiers effectivement lues, sans garantir un instantané atomique du dépôt.

Priorités : P1 = à traiter avant de valider le déploiement ou de sortir du laboratoire ; P2 = fiabilisation et qualité des preuves. Ces niveaux expriment une appréciation de revue, pas un score CVSS.

3. État des preuves disponibles

Élément	Observation et portée
Tests automatisés	data/test_summary.json : 14 tests, 0 échec, 0 erreur, datés du 19/10/2026 à 14:22:57 UTC. Résultats déclarés, non réexécutés pendant la revue.
Essai deux PC	data/live_2pc_validation.json : statut PASS, source 192.168.137.135 vers hôte 192.168.137.1, état RECOVERED, sonde et transaction T5 enregistrées. Preuve documentaire d'un essai antérieur, pas d'une exécution actuelle.
Docker	La dernière synthèse versionnée conserve le déploiement Docker au statut BLOCKED. La validation syntaxique Compose ne prouve pas la disponibilité applicative.
Livrables	PDF, PPTX, ZIP et manifeste présents dans deliverables/Workshop2026-M2-G2. Le manifeste daté du 21/10/2026 annonce 10 pages, 7 diapositives et l'absence de vidéo MP4. Les binaires ne sont pas validés indépendamment ici.

Point positif : les tests de bout en bout contrôlent une transaction initiale, une perturbation, l'évaluation ML, les actions de remédiation puis une transaction finale et l'invariant comptable. Limite : ils assemblent des applications Flask avec test_client et dispatchers locaux ; ils ne valident pas les communications et volumes réels de Docker.

La documentation EBIOS RM et PCA/PRA identifie le point de défaillance unique de l'hôte et distingue le prototype d'une continuité d'entreprise. Cette transparence est utile, mais plusieurs descriptions techniques et statuts restent à synchroniser avec le code et les preuves plus récentes.

Vue de priorisation

ID	Sujet	Priorité
F01	Base utilisée et persistance Docker	P1
F02	Injection des secrets et secrets par défaut	P1
F03	Intégration SOPS/Age non démontrée	P1
F04	Chaîne autonome ML en mode Docker	P1
F05	Port et routage du frontal Docker	P1
F06	Confiance dans l'IP déclarée par le client	P1
F07	Audit et anti-rejeu après redémarrage	P2
F08	Couverture du scan de secrets	P2
F09	Fiabilité du déploiement et des statuts PASS	P2
F10	Cohérence documentaire et des livrables	P2

4. Constats détaillés

F01 - Base réellement utilisée et persistance Docker | P1

Niveau de preuve : Confirmé dans la configuration et le code ; perte effective non testée.

Observation : Compose démarre PostgreSQL, monte pg_data et fournit DATABASE_URL à l'API. PaymentDatabase utilise exclusivement sqlite3 et résout son chemin depuis RESILIENCE_DB_PATH ou un chemin local. Le Dockerfile API fixe /app/data/payments.sqlite3. Aucun volume Compose n'est monté sur /app/data pour le service API.

Risque / conséquence : La présence de PostgreSQL ne protège pas les transactions de l'API. Les données SQLite sont écrites dans la couche modifiable du conteneur ; leur conservation au remplacement ou à la suppression n'est pas garantie. Un simple redémarrage du même conteneur doit être distingué de sa recréation. Docker documente que cette couche disparaît avec le conteneur [S1].

Recommandation : Choisir et documenter un moteur réellement supporté : implémenter la connexion PostgreSQL ou assumer SQLite avec un volume persistant. Ajouter une stratégie de sauvegarde et de restauration adaptée au moteur effectivement utilisé.

Critère de validation : Créer une transaction, vérifier le moteur et le support de stockage utilisés, recréer uniquement le conteneur API dans un environnement de test sauvegardé, puis relire la transaction, les soldes et la clé d'idempotence. Réaliser aussi une restauration sur une instance distincte.

Fichiers sources : services/api/database.py ; services/api/Dockerfile ; infra/compose/docker-compose.yml

F02 - Secrets chiffrés non injectés au lancement | P1

Niveau de preuve : Confirmé dans le chemin de lancement PowerShell examiné.

Observation : up.ps1 exécute verify, éventuellement bootstrap, puis docker compose sans injecter les secrets déchiffrés. Compose comporte des valeurs de repli pour JWT, HMAC, contrôle interne et mots de passe. La commande render-env ne produit pas les valeurs sensibles : elle affiche essentiellement les identifiants non secrets. .env.example contient des placeholders et plusieurs paramètres dont la prise en compte n'est pas établie.

Risque / conséquence : Une vérification réussie du coffre peut coexister avec l'utilisation de valeurs par défaut connues. La confidentialité des clés de signature et l'authentification interne ne sont alors pas assurées contre un acteur qui connaît ces valeurs et atteint les services concernés. Aucun usage malveillant effectif n'est établi.

Recommandation : Relier explicitement le coffre au mécanisme de déploiement ; refuser les secrets absents, placeholders et valeurs de laboratoire hors mode démo explicitement choisi. Prévoir la rotation et le redéploiement coordonné des consommateurs.

Critère de validation : Dans une instance isolée, démontrer que les anciennes valeurs ne permettent plus d'authentifier ou signer une requête et que les nouvelles valeurs fonctionnent. Ne jamais consigner les secrets eux-mêmes dans les journaux ou le rapport.

Fichiers sources : scripts/up.ps1 ; scripts/secrets_manager.py, main/render-env ; .env.example ; infra/compose/docker-compose.yml

4. Constats détaillés - suite

F03 - Qualification SOPS/Age de l’implémentation | P1

Niveau de preuve : Implémentation AES-GCM propre au projet confirmée ; interopérabilité non testée.

Observation : Le script utilise AESGCM et une clé symétrique aléatoire de 32 octets stockée en Base64 hors dépôt. Le destinataire age1 est construit à partir d’un SHA-256 tronqué. Le champ sops.age.enc contient le chiffrement d’une chaîne fixe, non un appel identifié au protocole Age. Une apparence ENC[AES256_GCM,...] ne démontre pas la compatibilité complète.

Risque / conséquence : Le chiffrement AES-GCM est bien présent ; ce n’est pas équivalent à démontrer l’usage standard de SOPS avec Age. Le code officiel SOPS décrit l’utilisation d’une clé publique Age pour chiffrer la clé de données [S3]. Une incompatibilité rendrait des procédures de reprise ou d’échange de clés standard inopérantes.

Recommandation : Utiliser les outils et bibliothèques SOPS/Age reconnus, ou désigner explicitement le mécanisme comme un coffre AES-GCM spécifique. Documenter la sauvegarde de la clé maîtresse, les permissions du fichier et la reprise sur un nouveau poste.

Critère de validation : Effectuer un cycle de chiffrement/déchiffrement avec les outils officiels si la compatibilité est revendiquée, puis une restauration indépendante et une rotation. Sans ces essais, je ne peux pas confirmer la compatibilité SOPS/Age.

Fichiers sources : scripts/secrets_manager.py : ensure_master_key, _derive_public_recipient, bootstrap_secrets ; documentation SOPS officielle

F04 - Chaîne autonome ML dans le déploiement Docker | P1

Niveau de preuve : Raccordement automatique absent du service détecteur et du Compose examinés ; lancement natif non intégralement audité.

Observation : Le détecteur expose /evaluate mais ne comporte pas de boucle de collecte périodique dans le code consulté. DETECTOR_AUTO_PUSH est false par défaut et Compose ne le définit pas. Les variables API_METRICS_URL et WAF_METRICS_URL de Compose ne sont pas exploitées par ce fichier. Les tests pilotent explicitement l’évaluation et la transmission au remédiateur.

Risque / conséquence : Un détecteur peut être sain selon /health sans analyser le trafic ni déclencher de remédiation. Le fonctionnement du lanceur natif ou des tests ne démontre pas celui du Compose seul.

Recommandation : Identifier un collecteur/ordonnanceur unique, le déclarer dans le déploiement et activer explicitement les échanges requis. Valider la fraîcheur des données, l’échec de livraison des alertes et les retours à la normale.

Critère de validation : Démarrer la pile Docker seule, sans navigateur ni lanceur natif ; constater l’augmentation des inférences, puis exécuter un scénario autorisé et borné jusqu’à la preuve de reprise. Tester aussi la panne du collecteur.

Fichiers sources : services/detector/app.py ; services/detector/Dockerfile ; infra/compose/docker-compose.yml ; tests/test_e2e_resilience.py

4. Constats détaillés - suite

F05 - Port et routage du frontal Docker | P1

Niveau de preuve : Incohérences de configuration confirmées ; disponibilité réseau non testée.

Observation : L'image finale est nginx:1.27-alpine sans configuration personnalisée copiée ou montée. Le Dockerfile déclare EXPOSE 3000 et Compose publie 3000:3000 ; le healthcheck vise aussi 3000. Nginx dans son image standard sert sur le port 80 [S2] ; EXPOSE ne configure pas le serveur. React utilise window.RESILIENCE_API_BASE ou une URL relative ; la variable VITE_API_BASE_URL fournie au lancement n'est pas lue par ce code. Les proxys Vite ne sont pas la configuration Nginx du build statique.

Risque / conséquence : Le port publié et le healthcheck peuvent viser un port sans serveur. Même une fois l'interface joignable, les appels relatifs /api, /auth, /soc et /rag ne disposent pas de relais Nginx identifié vers le WAF.

Recommandation : Aligner l'écoute Nginx, la publication et le healthcheck. Choisir un routage même origine vers le WAF ou un mécanisme explicite de configuration de l'URL API ; dans ce dernier cas, traiter également la politique CORS.

Critère de validation : Après construction propre, accéder au frontal par le port publié, authentifier un utilisateur, créer un paiement, lire le tableau SOC et interroger le RAG depuis un navigateur. Vérifier les requêtes réseau et les codes HTTP.

Fichiers sources : services/frontend/Dockerfile ; services/frontend/src/App.jsx ; services/frontend/vite.config.js ; infra/compose/docker-compose.yml

F06 - Attribution de l'adresse IP cliente | P1

Niveau de preuve : Confiance directe dans l'en-tête confirmée ; contournement non exécuté.

Observation : Le WAF et l'API prennent le premier élément de X-Forwarded-For lorsqu'il est fourni. Le WAF utilise cette valeur pour le quota et le blocage dynamique. Aucune vérification d'un proxy de confiance n'apparaît dans ce chemin ; le scénario de laboratoire fournit volontairement cet en-tête pour simuler une source.

Risque / conséquence : Un client pouvant choisir cet en-tête peut fausser les journaux et l'attribution de charge. Il peut potentiellement contourner les mesures par IP ou faire attribuer son trafic à un tiers. La protection des IP critiques de l'exécuteur ne résout pas l'authenticité de la source.

Recommandation : Distinguer strictement les modes simulation et réseau réel. À l'entrée directe, utiliser l'adresse de la connexion ; derrière un proxy maîtrisé, supprimer les en-têtes entrants non fiables et appliquer une politique de confiance précise.

Critère de validation : Comparer les résultats de requêtes depuis deux postes, avec et sans en-tête arbitraire. L'IP utilisée par la corrélation et la politique doit correspondre au client observé par la chaîne de confiance, non à une valeur librement choisie.

Fichiers sources : services/waf_gateway/app.py : _forward_request ; services/api/app.py : _before_request ; scenarios/controlled_attack.py

4. Constats détaillés - suite

F07 - Continuité du journal d’audit et de l’anti-rejeu | P2

Niveau de preuve : Réinitialisation de l’état mémoire confirmée ; altération de fichiers non testée.

Observation : L’exécuteur initialise _last_hash à zéro, _audit_entries à une liste vide et _seen_command_ids à un ensemble vide. Le journal est ajouté au fichier, mais le constructeur ne recharge pas l’historique. verify_audit_chain vérifie la liste en mémoire, pas le fichier persistant. Le contrôle puis l’enregistrement d’un command_id s’effectuent dans deux sections de verrou distinctes.

Risque / conséquence : Après redémarrage, une vérification valide peut ne couvrir que les nouvelles entrées ; une nouvelle chaîne peut être ajoutée après l’ancienne. Les identifiants anti-rejeu sont perdus, avec un risque résiduel pour un ordre encore dans la fenêtre de 30 secondes. L’atomicité en concurrence mérite aussi un test. Le terme « immuable » dépasse les garanties démontrées par le seul chaînage.

Recommandation : Recharger et vérifier le fichier au démarrage, conserver la tête de chaîne et rendre atomique la réservation d’un identifiant. Prévoir un ancrage externe ou un stockage à protection adaptée si une résistance à la réécriture est exigée.

Critère de validation : Exécuter des commandes, redémarrer l’exécuteur et vérifier toute la chaîne. Altérer une copie du fichier d’essai et vérifier la détection. Tester un rejeu encore frais et deux soumissions concurrentes d’un même identifiant.

Fichiers sources : services/executor/app.py : __init__, verify_audit_chain, execute_validated_command

F08 - Portée du scan de secrets | P2

Niveau de preuve : Limites des règles confirmées ; aucune nouvelle fuite recherchée de façon exhaustive.

Observation : Le scanner reconnaît notamment ghp_ mais ne contient pas de règle github_pat_. Il ignore .git et plusieurs formats binaires, dont ZIP, PDF et PPTX. Ses règles ne permettent pas de conclure à l’absence de tout secret arbitraire ; les valeurs de laboratoire codées en clair ne sont pas rejetées par ces motifs.

Risque / conséquence : Le statut PASS signifie uniquement qu’aucune règle implémentée n’a détecté de correspondance dans les fichiers parcourus. Il ne prouve pas l’absence de secrets dans l’historique ou les archives.

Recommandation : Élargir les formats reconnus, analyser l’historique avec un outil dédié et inspecter séparément les archives livrées. Définir une politique explicite pour les valeurs de démonstration et les placeholders.

Critère de validation : Utiliser uniquement des marqueurs synthétiques non fonctionnels pour vérifier la détection des différentes familles, de l’historique et des archives ; vérifier aussi que les journaux de scan ne révèlent pas de valeurs complètes.

Fichiers sources : scripts/secret_scan.py : SECRET_CONTENT_PATTERNS, IGNORED_DIRS, BINARY_EXTENSIONS

4. Constats détaillés - suite

F09 - Déploiement et fiabilité des statuts de validation | P2

Niveau de preuve : Défauts de chemins et raccourcis de validation confirmés dans les fichiers consultés.

Observation : Dans infra/ansible/playbooks/deploy.yml, playbook_dir désigne infra/ansible/playbooks. Les chemins déclarés vers le dépôt, Compose et secrets_manager.py ne remontent pas aux emplacements réels attendus. render-env est enregistré mais non injecté à Compose. Dans run_all_tests.py, le statut « déploiement » dépend de docker info, pas d'un déploiement réussi ; le statut deux-PC dépend de l'existence d'un fichier JSON. Le code de sortie final ne traite que le succès de la suite unittest.

Risque / conséquence : La présence du démon Docker, d'un document de preuve ou d'un YAML valide peut être confondue avec une recette applicative réussie. Un échec de validation complémentaire peut ne pas rendre le lanceur global non nul.

Recommandation : Corriger les chemins calculés, exécuter réellement le playbook sur un environnement propre et définir des critères séparés pour configuration, build, démarrage, santé et parcours métier. Rendre le statut final cohérent avec les contrôles bloquants.

Critère de validation : À partir d'un clone propre, exécuter le chemin de déploiement annoncé. Introduire volontairement un échec non destructif de contrôle et vérifier qu'il produit un statut FAIL et un code de sortie non nul.

Fichiers sources : infra/ansible/playbooks/deploy.yml ; tests/run_all_tests.py ; scripts/up.ps1

F10 - Cohérence documentaire et livrables | P2

Niveau de preuve : Divergences documentaires confirmées ; contenu des binaires non audité.

Observation : Le rapport de recette et la matrice décrivent encore le test deux-PC comme bloqué, tandis qu'un JSON plus récent et la synthèse de tests le déclarent réussi. Le générateur de dossier contient des affirmations fixes sur Docker actif alors que la synthèse versionnée indique BLOCKED. Le seuil RAG est 0,06 dans le code mais 0,12 dans plusieurs descriptions. Les documents décrivent aussi PostgreSQL, un reset de pool et certaines garanties plus fortes que le comportement observé.

Risque / conséquence : Un supérieur, un jury ou un opérateur peut prendre pour démontrée une capacité seulement préparée, testée localement ou annoncée. Les procédures RAG reposant sur ces documents peuvent propager ces divergences.

Recommandation : Établir une source unique des paramètres et résultats, lier chaque mesure au mode d'exécution et à une version, puis régénérer les rendus après la recette. Marquer explicitement les garanties hors périmètre et les essais non réalisés.

Critère de validation : Comparer README, matrice, rapport de tests, gouvernance, manifeste et rendus aux mêmes preuves datées. Vérifier le seuil RAG effectif, le moteur de base et les statuts Docker/deux-PC. Confirmer la présence et la conformité de la vidéo réelle, actuellement absente selon le manifeste.

Fichiers sources : docs/test-report.md ; docs/requirements-matrix.md ; docs/governance/pca-pra.md ; services/rag/app.py ; scripts/build_deliverables.py ; deliverables_manifest.json

5. Plan de traitement proposé

Étape	Travaux et responsable à désigner	Condition de clôture
A - Avant validation Docker	Responsable intégration + développement + infrastructure : F01, F04, F05.	Stockage vérifié après recréation ; interface complète ; chaîne automatique déclenchée par la pile seule.
B - Avant exposition élargie	Référent sécurité + développement : F02, F03, F06.	Secrets effectivement consommés, compatibilité ou qualification du coffre établie, IP source fiable.
C - Fiabilisation opérationnelle	Référent sécurité + exploitation + qualité : F07, F08, F09.	Audit relu après redémarrage ; tests de scan ; déploiement reproductible et statuts fiables.
D - Avant diffusion finale	Chef de projet + responsables de piliers : F10.	Documents et livrables synchronisés sur des preuves datées ; vidéo réelle disponible si requise.

Aucun délai chiffré ni coût n'est proposé sans estimation par les responsables techniques. Les corrections doivent être réalisées sur une branche dédiée, avec sauvegarde des données, tests de non-régression et procédure de retour arrière. Cette recommandation ne vaut pas autorisation de modifier le dépôt.

Décision attendue de l'encadrement

Désigner un responsable d'intégration, confirmer le moteur de données cible et le mode d'exécution de référence, puis conditionner l'acceptation technique à une recette reproductible. Tant que les P1 ne sont pas clôturés, présenter les résultats comme ceux d'un démonstrateur local et non d'une plateforme Docker intégralement validée.

6. Éléments positifs à préserver

Maintenir les montants entiers et les contraintes SQL ; l'idempotence dans la transaction ; le refus d'écriture pour l'auditeur ; l'exécuteur sans shell arbitraire et ses actions limitées ; la sonde de reprise transactionnelle plutôt qu'un simple contrôle /health ; les plafonds du scénario de charge ; la séparation conceptuelle entre RAG consultatif et décision de remédiation. Les performances ML restent à interpréter dans le périmètre synthétique du laboratoire, sans extrapolation à une production réelle.

Annexe - Traçabilité des sources

Les sources internes ci-dessous proviennent de la lecture authentifiée du dépôt. Les empreintes sont celles retournées pour le contenu des fichiers ; elles ne constituent pas un identifiant de commit commun. Les chemins cités dans chaque fiche complètent cet index.

services/api/database.py | SHA : ff2bc29c6cac45f4854e6ea81a47a02a71d3881a

services/api/Dockerfile | SHA : f9f1ab3511ca0be1bfd051ded56c7c6f0cd556ce

infra/compose/docker-compose.yml | SHA : bf8251a315053492976c0355eaf49ec5f0adf98d

scripts/secrets_manager.py | SHA : 2bcd2ba742433d541171b80d94aa54338fdd8682

scripts/up.ps1 | SHA : 169e995ae3cef0b8d4e238671fdcd709c18535a1

services/detector/app.py | SHA : 90cf3d7d5a2b3e158522aca7b28aae3bc79d1870

services/frontend/Dockerfile | SHA : 789e1e29db6a5261274a764efd77a441ce4db0a0

services/frontend/src/App.jsx | SHA : 58ee8452be66eabca83ad116410d52a62808a58e

services/waf_gateway/app.py | SHA : 4f829b931a3d9a0f7764440c6ae10df9443b43a0

services/executor/app.py | SHA : 17a63b2b1e2059881fc0b90a95fef391fd85b045

scripts/secret_scan.py | SHA : 947fcf5c090ef7cc62895acfc57ca0b212f4615c

infra/ansible/playbooks/deploy.yml | SHA : 902d2e9492ffb589c52e1b4eb4611501bef951be

tests/run_all_tests.py | SHA : 0d4bed2a720ee35e6e1a9944304ee984205f4c8c

data/test_summary.json | SHA : 44f122bf204d3e110577381d58c894a278f28402

data/live_2pc_validation.json | SHA : 9dca1281ed18b230625891edf8d7dc2b6b7df628

deliverables/Workshop2026-M2-G2/deliverables_manifest.json | SHA : 9307173fc79ef914f15828b3ae02f826d9eea62f

Documentation publique utilisée pour interpréter les constats

[S1] Docker Docs, Storage : les données de la couche modifiable ne persistent pas à la destruction du conteneur ; les volumes persistent indépendamment de son cycle de vie. Source publique consultée pendant la revue : documentation Docker « Storage ».

[S2] NGINX, Deploying NGINX and NGINX Plus with Docker : l'image et la configuration standard sont présentées avec le port conteneur 80. Source publique consultée pendant la revue : guide d'administration NGINX, déploiement Docker.

[S3] Dépôt officiel getsops/sops, fichier age/keysource.go : la clé publique Age chiffre la clé de données SOPS ; la clé chiffrée est enregistrée dans les métadonnées. Source publique consultée pendant la revue : implémentation officielle SOPS/Age.

Aucune valeur complète de secret, aucun jeton d'accès et aucun mot de passe de démonstration ne sont reproduits dans ce rapport.